

SEO PACKAGE

1. SEO Title

SQL WHERE Clause Explained: AND, OR, IN, BETWEEN, LIKE & the NULL Trap (Beginner to Confident)

2. SEO Subtitle

Master every filtering operator in SQL — and learn why `WHERE age > 21 OR 25` silently returns every row in your table.

3. Featured Quote

"The cheapest row to process is the one you filtered out at the door."

4. Meta Description

Learn the SQL WHERE clause the right way: AND, OR, NOT, IN, BETWEEN, LIKE, and the NULL trap. Avoid silent bugs and write faster, correct queries. (156 chars)

5. URL Slug

`sql-where-clause-and-or-in-between-like-null-explained`

6. Keywords

SQL WHERE clause, SQL filtering, AND OR NOT SQL, SQL IN operator, SQL BETWEEN, SQL LIKE wildcard, IS NULL SQL, SQL operator precedence, SQL query optimization, SQL for beginners, SQL tutorial, database filtering, SQL best practices, learn SQL, SQL interview questions, WHERE clause examples, SQL NULL trap, SQL predicate, relational databases, data analytics SQL

7. Tags

SQL, Databases, WHERE Clause, SQL Tutorial, Data Analytics, Coding Interview, SQL Operators, Learn To Code, Query Optimization, NULL Handling, SQL Beginners, Software Engineering, Data Structures, Backend Development, SQL Best Practices

8. Introduction

Every meaningful question you ask a database is really a filtering question. *Which* users signed up last month? *Which* orders are still unpaid? *Which* products are both cheap and in stock? The tables hold millions of rows, but you almost never want all of them. You want a slice — and the `WHERE` clause is the tool that carves out that slice.

This is Lesson 3 in the SQL series, and it's the one that separates people who "know some SQL" from people who can trust their own queries. That distinction matters because filtering bugs are the quiet kind. Your query runs. It returns rows. Nothing errors. But the rows are *wrong* — and in a report that decides budget, headcount, or who gets an email, silently wrong is far more dangerous than loudly broken.

Interviewers know this, which is why filtering questions show up constantly in SQL and data-analytics screens. They're rarely testing whether you can spell `SELECT`. They're testing whether you understand **operator precedence** (does `AND` bind tighter than `OR`?), whether you know that `NULL` breaks equality, and whether you can pick `IN` over five stacked `OR` s. These are the details that reveal whether you've actually shipped queries against real data.

By the end of this article you'll have a complete filtering toolkit — `AND`, `OR`, `NOT`, `IN`, `BETWEEN`, `LIKE` — plus a clear mental model for the two traps that catch almost everyone: precedence and `NULL`. You'll also understand *why* filtering early makes your queries faster, which is one of the most practical performance lessons in all of SQL.

9. Problem Overview

Picture a warehouse with a million boxes. You want only the red ones from last March. You do not want to walk every aisle yourself — you want to hand the warehouse a description and have it bring back exactly the matching boxes.

That is precisely what the `WHERE` clause does. `WHERE` is a **filter**: it evaluates a condition against every row and keeps only the rows where that condition is *true*.

Here's the mental model that makes everything click: **`WHERE` is a bouncer at the door**. Each row walks up. The bouncer checks it against a rule. If the row's answer is `true`, it's let into the result set. If

it's `false`, it's turned away. That's the whole job.

The condition you hand the bouncer is built from operators:

Operator	What it does	Effect on result size
AND	Both conditions must be true	Narrows (fewer rows)
OR	At least one must be true	Widens (more rows)
NOT	Inverts a condition	Excludes what you name
IN	Matches any value in a list	Shorthand for many OR s
BETWEEN	Matches a range, ends included	Continuous ranges
LIKE	Matches text patterns with wildcards	Fuzzy text search

The problem this lesson solves: **how do you describe exactly the rows you want — correctly and efficiently — without accidentally letting the wrong ones through?**

10. Example

Let's ground everything in one small table. Imagine a `users` table:

id	name	age	city	status	phone
1	Asha	17	Delhi	active	98xxxxxx01
2	Ravi	24	Mumbai	cancelled	NULL
3	Meena	30	Delhi	active	98xxxxxx03
4	Jason	19	Pune	active	NULL
5	Sara	45	Mumbai	active	98xxxxxx05

A simple filter:

```
SELECT name, age
FROM users
WHERE age > 18;
```

Output:

name	age
Ravi	24
Meena	30
Jason	19
Sara	45

Asha (17) is turned away at the door — her row answered `false` to `age > 18`. Everyone else is an adult, so they walk in. That's `WHERE` doing exactly one job: keep the `true` rows, drop the rest. We'll reuse this table throughout.

11. Intuition

Before touching more operators, sit with one idea: a **WHERE condition is a yes/no question asked of a single row at a time**. The database doesn't think about the whole table at once. It walks up to row 1 and asks, "Is your age greater than 18?" Then row 2. Then row 3. Every operator you're about to learn is just a richer way of phrasing that yes/no question.

Once you internalize "one row, one question, true or false," the combining operators stop being intimidating:

- **AND is two demands at once**. You're shopping and you insist the item be *cheap* **and** *in stock*. A \$50 sold-out item fails — it only passed one test. **AND** makes you pickier, so results shrink.
- **OR is the friendly opposite**. "I'll take coffee **or** tea, whichever you have." A row needs to satisfy only one side. **OR** opens the door wider.
- **NOT flips the question**. Instead of listing what you want, you point at what you don't and say "everything else."

Now the title's trap. Consider this broken query:

```
-- BROKEN: does not mean what it looks like
WHERE age > 21 OR 25
```

A newcomer reads this as "age greater than 21 or 25." But SQL doesn't read English. **OR** expects a full condition on both sides. `25` on its own isn't a comparison — most databases treat a nonzero number as **truthy**, so `age > 21 OR 25` collapses to `... OR true`, and `OR true` is always true. **Every row passes. The filter silently returns everyone.** What you meant was `age > 21 OR age > 25` (which is just `age > 21` anyway) or, more likely, `age IN (21, 25)`.

The experienced-engineer instinct here is: *when a filter returns way more rows than expected, suspect an operator that accidentally evaluates to always-true*. That instinct is what the rest of this article builds.

12. Brute Force Solution

Idea. Suppose you want users from Delhi, Mumbai, or Pune. The most literal, no-cleverness approach is to spell out every possibility with **OR**:

```
SELECT name, city
FROM users
WHERE city = 'Delhi'
```

```
OR city = 'Mumbai'  
OR city = 'Pune';
```

Algorithm. 1. For each row, test `city = 'Delhi'` . 2. If false, test `city = 'Mumbai'` . 3. If still false, test `city = 'Pune'` . 4. If any is true, keep the row.

Advantages. - Completely explicit — a beginner can read it top to bottom. - Works everywhere, no special syntax to remember.

Disadvantages. - Grows into a wall of text. Ten cities means nine `OR` s and lots of repetition. - Repeating the column name `city` invites typos (`ciyt = 'Pune'`). - The real danger: the moment you combine these `OR` s with an `AND` , precedence bites you (see the next section).

Complexity. Time is $O(n)$ — the database still examines each of the `n` rows once. Space is $O(1)$ beyond the output. The brute force here isn't slower asymptotically; it's slower to *read*, *write*, and *maintain*, and it's a bug magnet.

13. Optimal Solution

The optimal filter is the same logic expressed clearly, using the right operator for each shape of question, and guarding precedence with parentheses.

Use `IN` for a shortlist

When you're checking membership in a fixed list of separate values, `IN` is the clean answer:

```
SELECT name, city  
FROM users  
WHERE city IN ('Delhi', 'Mumbai', 'Pune');
```

This is **exactly equivalent** to the three- `OR` version — it just reads like plain English and names the column once. `IN` is genuinely underrated; people write five `OR` s where one `IN` would do.

Use `BETWEEN` for continuous ranges

When your values form a range — ages, prices, dates — `BETWEEN` says it best. **Both ends are inclusive**, a detail people forget:

```
SELECT name, age  
FROM users  
WHERE age BETWEEN 18 AND 30;    -- 18 and 30 both count
```

That's identical to `age >= 18 AND age <= 30`. Choose `IN` for a *scattered list*, `BETWEEN` for a *continuous range* — `BETWEEN` cannot express a scattered set, and `IN` cannot express "everything from 100 to 500."

Use `LIKE` for pattern matching

When you know only part of the text, `LIKE` fills the blanks:

- `%` — any run of characters (zero or more)
- `_` — exactly one character

```
SELECT name FROM users WHERE name LIKE 'A%';    -- starts with A
SELECT name FROM users WHERE name LIKE '%son';  -- ends with 'son' (Jason, Mason)
```

Guard precedence with parentheses

This is the fix for the bug that put minors in a Delhi report. Because `AND` is evaluated before `OR` (just like `×` before `+` in math), this query:

```
-- means: (age >= 18 AND city = 'Delhi') OR city = 'Mumbai'
WHERE age >= 18 AND city = 'Delhi' OR city = 'Mumbai';
```

quietly lets in *every* Mumbai row regardless of age. Wrap the `OR` group to say what you mean:

```
WHERE age >= 18
      AND (city = 'Delhi' OR city = 'Mumbai');
```

Now the age rule applies to both cities. **Rule of thumb: whenever `AND` and `OR` share a `WHERE` clause, parenthesize the `OR` part.**

Handle `NULL` with `IS NULL`

`NULL` means *unknown*, not zero and not empty. Unknown can never equal anything — not even another unknown. So `phone = NULL` matches **nothing**, silently:

```
-- WRONG: returns zero rows, always
WHERE phone = NULL

-- RIGHT
WHERE phone IS NULL      -- users with no phone
WHERE phone IS NOT NULL  -- users with a phone
```

14. Python Code

SQL is the star, but here's a small, clean Python model that mirrors exactly how the `WHERE` bouncer thinks — one row, one predicate, true or false — so you can see the semantics (including the `NULL` trap) in code you can run.

```

"""A tiny WHERE-clause simulator: filter rows by a predicate, SQL-style."""
from typing import Callable, Optional

# One row = one dict. `phone=None` models SQL NULL (unknown).
Row = dict

def where(rows: list[Row], predicate: Callable[[Row], bool]) -> list[Row]:
    """Keep only rows for which the predicate returns True (the bouncer)."""
    return [row for row in rows if predicate(row)]

def is_null(value: Optional[object]) -> bool:
    """SQL IS NULL: the only correct way to test for an unknown value."""
    return value is None

users = [
    {"name": "Asha", "age": 17, "city": "Delhi", "phone": "01"},
    {"name": "Ravi", "age": 24, "city": "Mumbai", "phone": None},
    {"name": "Meena", "age": 30, "city": "Delhi", "phone": "03"},
    {"name": "Jason", "age": 19, "city": "Pune", "phone": None},
    {"name": "Sara", "age": 45, "city": "Mumbai", "phone": "05"},
]

# WHERE age >= 18 AND (city = 'Delhi' OR city = 'Mumbai')
adults_in_two_cities = where(
    users,
    lambda r: r["age"] >= 18 and r["city"] in ("Delhi", "Mumbai"),
)

# WHERE phone IS NULL — NOT phone = None-as-equality
no_phone = where(users, lambda r: is_null(r["phone"]))

if __name__ == "__main__":
    assert [r["name"] for r in adults_in_two_cities] == ["Ravi", "Meena", "Sara"]
    assert [r["name"] for r in no_phone] == ["Ravi", "Jason"]
    # The NULL trap: equality against None-as-unknown finds nobody in SQL.
    # Here we prove why you must special-case it with is_null().
    print("All filters behaved correctly.")

```

15. Code Walkthrough

- `where(rows, predicate)` is the entire `WHERE` clause in one line. It walks each row, applies the predicate (the bouncer's rule), and keeps the `true` ones. This is the $O(n)$ scan a real database performs.
- `is_null(value)` exists to make the point explicit: testing for a missing value is *not* an equality check. In SQL, `phone = NULL` is wrong; `phone IS NULL` is right. Our helper uses `value is None` — Python's own "unknown" sentinel — to model that.
- `adults_in_two_cities` encodes the corrected, parenthesized filter. Note the Python `and` binds tighter than `or` too, but because we used `in ("Delhi", "Mumbai")` there's no ambiguity — which is exactly why `IN` prevents precedence bugs.
- The `assert`s are the self-check. If someone "fixes" the predicate incorrectly (say, drops the parentheses meaning), the asserts fail loudly instead of returning quietly wrong data — the same lesson the SQL bug teaches.

16. Dry Run

Let's trace `age >= 18 AND (city = 'Delhi' OR city = 'Mumbai')` against every row:

Row	age ≥ 18?	Delhi or Mumbai?	Both true?	Kept?
Asha (17, Delhi)	✗	✓	✗	No
Ravi (24, Mumbai)	✓	✓	✓	Yes
Meena (30, Delhi)	✓	✓	✓	Yes
Jason (19, Pune)	✓	✗	✗	No
Sara (45, Mumbai)	✓	✓	✓	Yes

Result: **Ravi, Meena, Sara**. Asha is dropped for being under 18 (the parentheses ensured the age rule applies even to Delhi/Mumbai rows), and Jason is dropped because Pune isn't in the list. Compare this to the *buggy* unparenthesized version, which would have kept Ravi, Meena, Sara **and** any under-18 Mumbai user — the silent bug.

17. Complexity Analysis

- **Time: $O(n)$.** A `WHERE` clause without an index examines each of the `n` rows once and tests the predicate. `IN`, `BETWEEN`, and `LIKE` don't change that ceiling by themselves — they're different ways to phrase the per-row test, each roughly constant work per row.

- **Space: O(1)** for the evaluation itself (beyond the output rows you return). The database evaluates row by row and doesn't need to hold the whole table's booleans in memory.
- **Why this is correct:** filtering is inherently a single pass — you must look at a row to decide whether to keep it, and you never need to look twice. The important nuance is that a good `WHERE` clause plus an **index** can push this below O(n) in practice, because the index lets the engine skip rows it knows can't match. That's the deeper reason filtering early is fast.

18. Alternative Solutions

OR vs IN vs BETWEEN — same result, different fit:

Approach	Best when	Weakness
Stacked <code>OR</code>	You truly need mixed conditions on <i>different</i> columns	Verbose, error-prone, precedence traps
<code>IN (list)</code>	A specific shortlist of separate values (three cities)	Awkward for continuous ranges
<code>BETWEEN a AND b</code>	A continuous range (prices, dates, ages)	Can't express a scattered set

NOT IN vs NOT EXISTS : for exclusion, `WHERE city NOT IN (...)` reads cleanly — but beware: if the list contains a `NULL`, `NOT IN` can return zero rows unexpectedly (another `NULL` trap). `NOT EXISTS` sidesteps that. Reach for it when your exclusion list might contain unknowns.

LIKE vs full-text search: `LIKE '%term%'` can't use a normal index (the leading `%` defeats it), so on large tables a dedicated full-text index outperforms it. `LIKE` is perfect for small tables and simple prefix matches like `'A%'`.

19. Edge Cases

- **Empty input:** filtering an empty table returns an empty result — no errors, the bouncer just has no one to check.
- **NULL values:** `= NULL` matches nothing; you must use `IS NULL` / `IS NOT NULL`. This is the single most common silent bug.
- **Duplicates:** `WHERE` doesn't deduplicate. Two identical qualifying rows both pass; use `DISTINCT` if you need uniqueness.
- **Boundary values in BETWEEN :** both endpoints are *included*. `BETWEEN 100 AND 500` keeps 100 and 500. If you meant to exclude them, use `>` and `<`.
- **Case sensitivity in LIKE :** behavior varies by database and collation. `'a%'` may or may not match `Asha`. Don't assume — check your engine.

- **Truthy literals:** `WHERE col > 21 OR 25` (a bare number) can evaluate to always-true and return everything.

20. Common Mistakes

1. **Using `= NULL` instead of `IS NULL`**. Returns zero rows, silently, every time. Always test unknowns with `IS NULL / IS NOT NULL`.
2. **Mixing `AND` and `OR` without parentheses**. `AND` binds tighter, so `a AND b OR c` means `(a AND b) OR c`. Wrap the `OR` group.
3. **Writing a bare value after `OR`**. `age > 21 OR 25` isn't `age IN (21, 25)`; the loose `25` can make the whole filter always-true.
4. **Forgetting `BETWEEN` is inclusive**. Off-by-one on both ends when you assumed the boundaries were excluded.
5. **Stacking five `OR` s where one `IN` belongs**. Not wrong, but harder to read and easier to typo — and it hides precedence risk.
6. **Leading-wildcard `LIKE '%x%'` on a huge table**. It can't use the index and scans everything; slow at scale.

21. Interview Questions

- What does `WHERE salary > 5000 OR 6000` return, and why is that dangerous?
- Explain operator precedence between `AND` and `OR`. Rewrite an ambiguous filter with parentheses.
- Why does `WHERE phone = NULL` return no rows? How do you correctly find rows with missing phones?
- When would you choose `IN` over `BETWEEN`, and vice versa?
- Is `BETWEEN 10 AND 20` inclusive or exclusive of the endpoints?
- Why might `col NOT IN (subquery)` return zero rows unexpectedly?
- How does filtering early in a `WHERE` clause improve query performance?
- Can `LIKE '%abc'` use an index? Explain.

22. Similar Problems

- **LeetCode 595 — Big Countries:** pure `WHERE` with `OR / AND` on two conditions; ideal precedence practice.
- **LeetCode 584 — Find Customer Referee:** the `NULL` trap in the wild — you must handle `referee_id IS NULL` alongside a value check.

- **LeetCode 183 — Customers Who Never Order:** exclusion logic (`NOT IN` / `NOT EXISTS`), directly related to `NOT` and the `NULL` -in-list gotcha.
- **LeetCode 1148 — Article Views I:** filtering with equality plus `DISTINCT` , reinforcing that `WHERE` doesn't dedupe.

Each one drills a specific muscle from this lesson: combining conditions, handling `NULL` , and excluding rows correctly.

23. Key Takeaways

- `WHERE` is a bouncer: one row, one yes/no question, keep the `true` ones.
- `AND` narrows, `OR` widens, `NOT` excludes.
- `IN` is a clean shortlist; `BETWEEN` is an inclusive range; `LIKE` is text search with `%` (many) and `_` (one).
- `AND` beats `OR` in precedence — always parenthesize the `OR` group.
- `NULL` means *unknown*; use `IS NULL` , never `= NULL` .
- Filtering early is a performance win: the cheapest row to process is the one you threw out at the door.

24. Watch the Video

Reading the logic is one thing — *seeing* the bouncer turn rows away makes it stick. The full walkthrough, with live queries and the exact bug that put minors in a Delhi report, is right here: [{LINK}](#). Watch it once before your next SQL interview and the precedence and `NULL` traps will never catch you off guard.

25. About the Series

This is part of **Fun with Learning Technology** — a series that takes the topics developers are quietly unsure about and makes them obvious. We build intuition first, code second, always explaining *why* an approach works rather than asking you to memorize syntax. Whether you're prepping for a coding interview, leveling up your data-analytics skills, or just tired of queries that "run but lie," the series meets you where you are and takes you one confident step further.

26. Call To Action

If this cleared up the `WHERE` clause for you, here's how to help the series grow: - 👍 **Like** the video on YouTube — it genuinely helps more developers find it. - 🔔 **Subscribe** so you don't miss the next SQL lesson. - 📧 **Subscribe to the Substack** for the written deep-dives like this one. - 💬 **Comment** with the filtering bug that once bit you. - 💻 **Share** it with a teammate who still writes `= NULL` .

SOCIAL MEDIA

LinkedIn Post

Here's a SQL bug that runs perfectly, returns rows, and is completely wrong:

```
WHERE age > 21 OR 25
```

A newcomer reads that as "age is 21 or 25." SQL doesn't read English. `OR` needs a full condition on both sides — a bare `25` is treated as truthy, so the whole filter collapses to "always true." Every row passes. Your report silently returns everyone.

This is the real skill in SQL filtering: not memorizing operators, but understanding the two traps that catch almost everyone.

Trap 1 — precedence. `AND` is evaluated before `OR` , just like $\times$ before $+$ in math. So `age >= 18 AND city = 'Delhi' OR city = 'Mumbai'` quietly lets in every Mumbai row regardless of age. The fix is one habit: whenever `AND` and `OR` share a `WHERE` clause, wrap the `OR` group in parentheses.

Trap 2 — NULL. `NULL` means *unknown*, not zero and not empty. Unknown can never equal anything, so `phone = NULL` matches nothing — silently. To find missing values you must write `IS NULL` .

The rest of the toolkit is friendlier: `AND` narrows, `OR` widens, `NOT` excludes, `IN` checks a shortlist, `BETWEEN` covers an inclusive range, and `LIKE` searches text with wildcards.

And there's a performance payoff. A good `WHERE` clause tells the database to throw out rows before the heavy sorting and grouping. The cheapest row to process is the one you filtered out at the door.

If you write SQL for reports or dashboards, these two traps are worth 10 minutes of your attention. Which one has bitten you? 🙅

SQL #DataAnalytics #SoftwareEngineering

#Databases #CodingInterview

Twitter/X Thread

1/ This SQL query runs, returns rows, and is 100% wrong:

```
WHERE age > 21 OR 25
```

It silently returns your ENTIRE table. Here's why — and the filtering traps every developer should know. 🧐

2/ Think of WHERE as a bouncer. Each row walks up, the bouncer checks it against a rule. True → walk in. False → turned away. Every operator is just a different way to phrase that yes/no question.

3/ The basics: • AND = two demands at once → narrows results • OR = either one works → widens results • NOT = everything EXCEPT what you name

4/ So why does `age > 21 OR 25` break? `OR` needs a full condition on both sides. A bare `25` is treated as `truthy` → `... OR true` → always true. Every row passes. Silently.

5/ The bigger trap: precedence. `AND` is evaluated BEFORE `OR` — just like $\times$ before $+$ in math.

6/ So this: `age >= 18 AND city = 'Delhi' OR city = 'Mumbai'` actually means `(age >= 18 AND Delhi) OR Mumbai`. It lets in under-18 Mumbai users you never wanted.

7/ The fix is one habit: when AND and OR share a WHERE clause, wrap the OR group in parentheses.

```
age >= 18 AND (city = 'Delhi' OR city = 'Mumbai') ✓
```

8/ Cleaner operators worth knowing: • IN → a shortlist: `city IN ('Delhi', 'Mumbai')` • BETWEEN → an inclusive range (both ends count!) • LIKE → text search: `%` = any chars, `_` = one char

9/ The sneakiest bug of all: NULL. It means *unknown*, not zero. Unknown can't equal anything — so `phone = NULL` matches NOTHING. To find missing values you MUST write `IS NULL`.

10/ And filtering isn't just correctness — it's speed. A good WHERE throws out rows before the heavy sorting begins. The cheapest row to process is the one you filtered out at the door. Full lesson → {LINK}

Facebook Post

Ever written a SQL query that ran fine but gave you way too many rows — and you couldn't figure out why? 🤔

There's a classic culprit: mixing AND and OR without parentheses. SQL evaluates AND before OR (just like times before plus in math), so your filter can quietly mean something totally different from what you typed — like sneaking under-18 users into an "adults only" report.

In this lesson we walk through the whole filtering toolkit — AND, OR, NOT, IN, BETWEEN, LIKE — plus the famous NULL trap (spoiler: `= NULL` matches nothing, ever). Clear examples, no jargon, and the *why* behind every rule. Perfect if you're learning SQL or brushing up for an interview. Come check it out! 📌 {LINK}

Reddit Summary

The WHERE clause traps that cause silently-wrong SQL (a beginner-friendly writeup)

Sharing a breakdown of SQL filtering aimed at people who "know some SQL" but occasionally get results they can't explain. The focus is on the two things that cause *silent* bugs — queries that run and return rows but are wrong.

Main points:

- **Operator precedence.** `AND` is evaluated before `OR`. So `age >= 18 AND city = 'Delhi' OR city = 'Mumbai'` parses as `(age >= 18 AND Delhi) OR Mumbai`, which includes every Mumbai row regardless of age. Fix: parenthesize the OR group whenever AND and OR mix.
- **The OR <bare value> gotcha.** `WHERE age > 21 OR 25` doesn't mean `IN (21, 25)`; the loose `25` is truthy and can make the whole filter always-true.
- **NULL is "unknown," not zero.** `col = NULL` matches nothing. You need `IS NULL` / `IS NOT NULL`. `NOT IN (subquery-with-nulls)` has a related failure mode worth knowing.
- **Operator fit:** `IN` for scattered shortlists, `BETWEEN` for continuous inclusive ranges, `LIKE` for patterns (`%` = many chars, `_` = one). Also a note that leading-wildcard `LIKE '%x%'` can't use an index.

Includes a table dry-run showing exactly which rows survive each filter, and the performance angle: filtering early lets the engine discard rows before sorting/grouping. Happy to discuss edge cases in the comments — especially the NULL-in- `NOT IN` one, which trips up a lot of people.

GITHUB README

SQL Lesson 3 – The WHERE Clause & Filtering Operators

> Master `AND`, `OR`, `NOT`, `IN`, `BETWEEN`, `LIKE`, and the `NULL` trap – and
> learn why `WHERE age > 21 OR 25` silently returns every row.

Problem

Databases hold millions of rows, but you almost never want all of them. The `WHERE` clause filters a table down to exactly the rows you care about. The challenge isn't the syntax – it's avoiding ****silently wrong**** filters: queries that run without error but return the wrong rows.

Intuition

`WHERE` is a bouncer at the door. Each row is checked against a condition. If the condition is `true`, the row is kept; if `false`, it's dropped. Every operator is just a different way to phrase that yes/no question:

Operator	Meaning	Effect
-----	-----	-----
`AND`	Both conditions true	Narrows
`OR`	At least one true	Widens
`NOT`	Invert a condition	Excludes
`IN`	Value is in a list	Shortlist
`BETWEEN`	Value in a range (**ends included**)	Range
`LIKE`	Text pattern (`%` many, `_` one)	Fuzzy text

Approach

Two rules prevent the most common bugs:

- **Precedence:**** `AND` is evaluated before `OR`. Always parenthesize the `OR` group when mixing them.

```
```sql
-- Wrong: (age >= 18 AND Delhi) OR Mumbai -> lets in under-18 Mumbai rows
WHERE age >= 18 AND city = 'Delhi' OR city = 'Mumbai';

-- Right
WHERE age >= 18 AND (city = 'Delhi' OR city = 'Mumbai');
```
```
- **NULL is unknown:**** `= NULL` matches nothing. Use `IS NULL` / `IS NOT NULL`.

Python Solution (WHERE simulator)

```
```python
from typing import Callable, Optional

Row = dict
```

```
def where(rows: list[Row], predicate: Callable[[Row], bool]) -> list[Row]:
 """Keep rows where the predicate is True – the WHERE bouncer."""
 return [row for row in rows if predicate(row)]

def is_null(value: Optional[object]) -> bool:
 """SQL IS NULL: the correct test for an unknown value."""
 return value is None

users = [
 {"name": "Asha", "age": 17, "city": "Delhi", "phone": "01"},
 {"name": "Ravi", "age": 24, "city": "Mumbai", "phone": None},
 {"name": "Meena", "age": 30, "city": "Delhi", "phone": "03"},
 {"name": "Jason", "age": 19, "city": "Pune", "phone": None},
 {"name": "Sara", "age": 45, "city": "Mumbai", "phone": "05"},
]

adults = where(users, lambda r: r["age"] >= 18 and r["city"] in ("Delhi", "Mumbai"))
no_phone = where(users, lambda r: is_null(r["phone"]))

assert [r["name"] for r in adults] == ["Ravi", "Meena", "Sara"]
assert [r["name"] for r in no_phone] == ["Ravi", "Jason"]
```

## Complexity

### Aspect Value

### Why

Time	$O(n)$	Each row is tested once (indexes can push below $O(n)$ )
Space	$O(1)$	Row-by-row evaluation, no full-table buffering

## Video

 Watch the full lesson: [{LINK}](#)

## Article

 Full written deep-dive with dry runs, edge cases, and interview questions: part of the **Fun with Learning Technology** series. ``

## Quick Review



**SQL filtering: when do you reach for IN vs a range vs a pattern?**

IN for a fixed shortlist of values, BETWEEN for a continuous low-to-high range (inclusive), LIKE for fuzzy text with % or \_ wildcards.

**Time cost of a WHERE filter on an unindexed column?**

$O(n)$  — a full table scan checks every row. An index on the filtered column drops lookups toward  $O(\log n)$ .

**Extra space used by a WHERE clause?**

$O(1)$  for the predicate itself; the engine tests row by row. Output storage is  $O(k)$  for the  $k$  matching rows returned.

**Trickiest filtering bug with NULL?**

NULL fails =, <>, IN, and NOT IN comparisons (returns unknown, not true). Use IS NULL / IS NOT NULL; NOT IN with any NULL yields no rows.

**AND vs OR mixed in one WHERE — the gotcha?**

AND binds tighter than OR, so it evaluates first. Wrap OR groups in parentheses, e.g. WHERE a AND (b OR c), or you get wrong rows.

**Interview: why filter in WHERE instead of after fetching?**

WHERE cuts rows at the server before transfer and aggregation, shrinking data processed and moved — faster, cheaper, and enables index use.

---

sql Lesson 3: Filtering — AND, OR, NOT, IN, BETWEEN, LIKE (Filtering allows you to subset data by defining logical criteria that each record must satisfy to be included in your results. By using specialized operators, you can build precise queries that isolate specific business entities or temporal ranges from massive datasets. This skill is essential for performance optimization and generating accurate reports, as it minimizes the volume of data processed by the server.) — free Python cheat sheet